CHAPTER 01 / FOUNDATIONS

Introduction to Snowflake and Modern Data Engineering

From raw measurements to a trustworthy decision

At 08:00, an energy analyst opens a dashboard that shows electricity generation in Finland. The chart is current, its labels look convincing, and the query returned successfully. But the source has retried yesterday's data, two timestamps have been interpreted in local time rather than UTC, and a missing quarter-hour has quietly disappeared from the total. The platform did what it was asked to do; the information is still wrong.

This is why modern data engineering is not primarily the art of moving bytes. It is the discipline of turning uncertain, changing inputs into *reliable data products* that people can interpret and use. Snowflake can provide a powerful environment for that work, but architecture, semantics, quality rules, and operational judgment remain the engineer's responsibility.

The central question of this chapter
How do we choose a data architecture that delivers correct, timely, maintainable answers without paying for complexity that our users do not need?

By the end of Chapter 1, you will be able to:

- Explain the evolution from traditional warehouses to cloud-based data platforms.
- Distinguish warehouses, lakes, lakehouses, and transactional stores by workload.
- Explain the role of Snowflake's storage, compute, and cloud-services layers.
- Compare batch, micro-batch, and streaming with measurable freshness requirements.
- Draft the first requirements, metrics, and context diagram for the Finland Energy Analytics capstone.

Prerequisites: no Snowflake experience; elementary SQL is helpful. The guided lab can be completed offline without a paid cloud account. The architecture discussion draws on Snowflake documentation [1]; all version-dependent features require verification before publication.

Reader's route Sections 1.1–1.3 build a historical and architectural mental model; 1.4–1.7 test engineering decisions; 1.8 introduces the capstone; 1.L turns the ideas into a reviewable specification. We will implement the platform incrementally in later chapters.

1.1 The evolution of modern data platforms

1.1.1 On-premises systems and traditional data warehouses

Imagine a manufacturer with order data inside an enterprise resource planning database, meter readings in CSV files, and finance reports in spreadsheets. Ten years ago, a common response was to operate a dedicated analytical database: extract records overnight, clean and reshape them, and load the results into a central warehouse. Analysts queried that carefully managed copy rather than overloading the system that processed orders.

That architecture solved a real problem: an *operational* system is optimized for timely, correct transactions, while analytical questions often scan many records, join histories, and compute aggregates. Separating them protects the daily business from an expensive reporting query. Dimensional models further help readers understand what a number means: "electricity delivered" requires a defined measure, grain, unit, time basis, and source.

Traditional warehouses were not a mistake. They were designed around understandable constraints: hardware procurement, fixed storage capacity, scheduled maintenance, limited processor headroom, and teams that had to decide in advance how much infrastructure to buy. Their weaknesses appeared when workload shapes changed faster than procurement cycles. A monthly reporting peak might require hardware that remained idle for the rest of the month.

The architectural distinction is not old versus new. It is *which responsibilities the system centralizes.* Even on cloud infrastructure, a poorly designed pipeline can reproduce every problem of its on-premises predecessor: stale extracts, undocumented transformations, and no recovery plan.



Figure 1.1. A familiar scheduled warehouse pattern. The issue is not nightly loading itself, but whether its delay and failure behavior meet the reporting contract.

Three questions turn the historical model into engineering judgment: *How fresh must the result be? Can the operational workload safely serve analysis? What happens if the extract runs twice or fails halfway through?* Those questions matter regardless of the vendor.

Explore the source project: P001: Modern Data Engineering with Snowflake [GitHub]. *External reference; not executed or licensed for reproduction as part of this chapter.*

1.1 From fixed capacity to elastic analytics

1.1.2 Elastic cloud analytics and separated workloads

Cloud platforms changed the procurement problem. Rather than purchase a machine sized for the year's busiest afternoon, a team can provision compute for a job and release it after the work ends. Snowflake applies this model through *virtual warehouses*: compute clusters can be assigned to different analytical activities while accessing a common data platform. Snowflake documents three architectural layers—database storage, compute, and cloud services—which we examine in detail in Chapter 2 [1].

The separation matters because analytical workloads compete differently. A data transformation job may scan and update large tables; a dashboard may issue many smaller queries during working hours; an experimental Python notebook may be unpredictable. Assigning distinct virtual warehouses can reduce contention between these compute workloads. It does not eliminate every shared dependency, and it does not make credits free: warehouses consume credits while running, subject to billing rules and minimums [2].

1.1.3 Evolving responsibilities of the data engineer

Infrastructure automation also shifts work rather than removing it. Modern data engineers must design data contracts, select ingestion modes, reason about schema changes, organize testing, secure access, monitor freshness, and explain costs. They ask whether a reported number can be traced back to its source and whether a corrected upstream record will update a downstream metric safely.

Worked decision: two teams, one dataset
The energy-analysis team refreshes a daily report once each morning; an operations team investigates current power measurements throughout the day. Both can use a shared governed dataset, but the same refresh target and compute schedule need not fit both. The engineering decision is to separate *service objectives* before selecting services or warehouse sizes.

In Snowflake, "elastic" is an engineering affordance, not a performance guarantee. Starting more compute might reduce queueing, but it will not repair an incorrect join or a duplicated source file. Likewise, a high-throughput platform will not resolve the meaning of an energy statistic. Correctness begins with agreement on identifiers, units, boundaries, and acceptable delays.

Decision checkpoint. Suppose reporting needs only yesterday's validated totals. A scheduled batch is a defensible default. If a control room requires updates within a short interval, revisit that design with an explicit freshness target and failure budget. You should justify the extra moving parts in operational terms, not because "streaming" sounds more modern.

Common failure. Teams often equate successful pipeline execution with accurate data. A green job status proves a process completed according to its controls; it does not prove the business meaning of the result. We will add independent invariants and reconciliation checks later.

1.2 Data warehouses, lakes, lakehouses, and data clouds

1.2.1 Data warehouses and data lakes

A *data warehouse* organizes information for repeatable analytical queries. Curated tables, stable dimensions, explicit metric definitions, and well-understood access patterns make it appropriate for business intelligence. A *data lake* is typically an object-storage-based collection of data in diverse formats and stages of refinement; it can accommodate raw files cheaply, but a folder of files is not by itself a governed, reliable analytics system.

The distinction is functional, not a claim that warehouses can store only tables or that

lakes cannot support analytics. Snowflake accepts structured and semi-structured data, while modern engines can query files in external storage. The critical question is who manages transaction semantics, metadata, security, transformation, and performance.

1.2.2 Lakehouse and data-cloud concepts

A *lakehouse* aims to combine open or lake-style storage with table metadata, reliable reads, and analytical processing. Apache Iceberg is one table format used in this ecosystem. Snowflake documents both Snowflake-managed storage and external-volume arrangements for Iceberg tables; those choices have different governance and operational implications [1, 3]. "Data cloud" is broader platform language describing governed access, processing, and collaboration around data; it is not a precise replacement for a physical architecture diagram.

1.2.3 Selecting architecture by workload rather than marketing labels

Approach	Useful when	Question to test
Warehouse	Curated BI, governed SQL metrics	Are model and query contracts stable?
Lake	Diverse raw files and replay history	Who enforces usable table semantics?
Lakehouse	Shared table formats and multiple engines	How are catalogs and writes coordinated?

For our energy case, retain an auditable source snapshot and build trustworthy analytical tables. Whether the source snapshot stays in an external object store, in Snowflake-managed storage, or in both is a cost, governance, and recovery decision. It is not dictated by the word "lakehouse."

Failure to avoid: assuming an open table format means every engine supports identical semantics, security, and performance. Verify catalog behavior, write ownership, compatibility, and cross-tool tests before committing to a multi-engine architecture.

Explore the source project: P002: Getting Started with Iceberg Tables [GitHub]. *External reference; not executed or licensed for reproduction as part of this chapter.*

1.3 OLTP versus OLAP: workloads and design choices

1.3.1 Transactional versus analytical access patterns

An electricity billing application may need to update one customer's payment atomically. An analyst may ask for two years of hourly electricity generation by source, region, and season. The first resembles *online transaction processing* (OLTP): short operations, selective reads, concurrent updates, and carefully enforced invariants. The second resembles *online analytical processing* (OLAP): scans, aggregates, joins, and historical comparisons. Both are databases; their principal workloads differ.

1.3.2 Read/write characteristics and latency

A workload specification should record operation mix, concurrency, data volume, read and write patterns, and acceptable latency. "Real time" is not a measurable target. "An event

must appear in the operational display within 60 seconds for 95% of accepted events" is closer, although it still needs a definition of event time and acceptance time.

Snowflake standard tables are primarily oriented toward analytics. Snowflake also documents *hybrid tables*, designed for lower-latency indexed reads and writes with transactional capabilities, but their support and limitations depend on cloud region and account features [1, 4]. The existence of a transactional-oriented option does not imply that Snowflake should replace every operational database.

1.3.3 Selecting a store for energy-meter use cases

For the running project, imagine two independent applications. A device-registration system changes individual meter records throughout the day, while an analytical platform calculates generation trends and power-to-energy aggregates across many readings. We would ordinarily evaluate a transactional system for registration state and a governed analytical model for historical analysis, with an explicit ingestion boundary between them.

Why granularity matters
A row recording 480 MW of average power for a 15-minute interval contributes $480 \times \frac{15}{60} = 120$ MWh of energy for that interval under the stated sampling assumption. Summing MW readings alone does *not* yield energy. An architecture that returns queries quickly but confuses those units is not successful.

Decision checkpoint. Describe a requirement for a single-record correction and another for a two-year trend query. What write behavior, consistency guarantees, latency, and history will each need? Write your answers before naming a database.

Common failure. Running a broad reporting query directly against the transactional application during peak business hours can harm its primary duty. Isolate and govern analytical access unless a measured workload justifies otherwise.

Explore the source project: P003: Hybrid Tables Performance Testing [GitHub]. *External reference; not executed or licensed for reproduction as part of this chapter.*

1.4 Snowflake ecosystem and core use cases

1.4.1 Core platform components and SQL/Python roles

Snowflake is a managed cloud data platform rather than software that you install on a local server. Its database storage layer manages persistent analytical data; virtual warehouses supply compute for supported SQL and Snowpark workloads; cloud services coordinate authentication, metadata, parsing, optimization, and other control functions [1]. Cloud-managed does not mean responsibility-free. The customer still defines permissions, trustworthy models, retention choices, and spending controls.

SQL is a first-class language for transformations and analysis. Python enters through connectors, Snowpark, and external tooling, among other workflows. Choosing one does not exclude the other: Python may retrieve a licensed API extract and validate it, while SQL defines the repeatable analytical model. Later chapters will distinguish where code executes and which component pays for it.

1.4.2 Structured versus semi-structured processing

An energy record may have a timestamp, a source category, and power measurement:

```
timestamp_utc / source_type / power_mw
```

An API may instead return nested JSON: a time stamp, several measurements, metadata, and quality flags inside one object. Snowflake can process structured and semi-structured data [1], but accepting flexible JSON is not the same as validating its semantics. Explicit checks are needed for missing values, units, time zones, and schema drift.

1.4.3 Business intelligence, data sharing, and AI use cases

Three broad patterns are relevant here: prepare stable data for BI, share governed data products with authorized consumers, and apply forecasting or other models to curated features. Snowflake supports sharing selected database objects with other Snowflake accounts, using provider-controlled shares rather than naïvely exporting a separate copy in every intra-platform case [5]. Neither this feature nor AI capability eliminates privacy reviews, contract requirements, or the need to measure correctness.

A small, defensible initial platform
`Public energy source` → `raw observations` → `validated energy intervals` → `daily metrics` → `BI report`. Start with this traceable path. Add continuous loading, data sharing, or AI only when they serve an explicit requirement.

Failure to avoid. Treating "supported by the platform" as "available in every edition and region." Product surfaces and pricing change; use official documentation, account-specific verification, and a recorded test date.

Explore the source project: P004: Zero to Snowflake [GitHub]. *External reference; not executed or licensed for reproduction as part of this chapter.*

Explore the source project: P005: Data Apps Demo [GitHub]. *External reference; not executed or licensed for reproduction as part of this chapter.*

1.5 The end-to-end data engineering lifecycle

1.5.1 Source discovery and data contracts

A useful data product begins with people and decisions rather than tables. Who owns the source? What can a "missing" value mean? How often is the feed updated? Which conditions make a record acceptable? A *data contract* captures such promises: schema, grain, units, identity, timeliness, provenance, and expected quality. A public API that returns JSON is only an entry point. Its schema and licensing terms must be recorded before the team builds dashboards around it.

1.5.2 Ingestion, transformation, and validation

The lifecycle then moves through ingestion, preservation of raw evidence, validation, transformation, and publication. Treat each boundary as a place where meaning can change. Parsing a timestamp can erase its offset. Filtering nulls can alter totals. Joining two tables at incompatible grains can multiply records. A reproducible pipeline makes such decisions

inspectable.

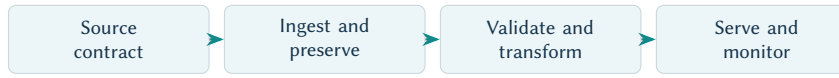


Figure 1.2. Data engineering is a lifecycle of contracts, transformations, tests, and responsibility—not a one-way file copy.

1.5.3 Monitoring, quality, and maintenance

Operational ownership begins after deployment. Useful indicators include latest source timestamp, delay to publication, rejected-record count, test failure rate, and cost by workload. A successful query is only one observation. If the source stops sending readings, yesterday's results may still look plausible; a freshness alert detects the silent failure.

Six questions for every stage
What went in? What came out? Which invariants hold? Which records were rejected? Can the step be repeated safely? Who responds when it fails? A team that can answer these questions is closer to operating a data product than a team that merely has a collection of scripts.

For a daily energy report, the first useful contract might be: every accepted interval is uniquely identified by source and UTC timestamp; power is expressed in MW; energy calculations respect interval length; no dashboard is marked "complete" until required quality checks pass. These are proposed *project requirements*, not claims about a public dataset.

Common failure. Monitoring only pipeline status ignores source freshness and business correctness. A row count can look normal even when all new rows are duplicates.

Explore the source project: P006: Snowflake Northstar Data Engineering [GitHub]. *External reference; not executed or licensed for reproduction as part of this chapter.*

1.6 Batch, streaming, and analytics architectures

1.6.1 Batch, micro-batch, and streaming models

A batch loads a bounded set of data at scheduled or deliberate times. A micro-batch collects smaller intervals, often frequently. Streaming receives records or events continuously and typically involves state, delivery, ordering, and recovery concerns. These describe different processing patterns; they are not interchangeable promises about end-to-end latency.

Snowflake supports bulk file loading with `COPY INTO`, file-oriented continuous loading with Snowpipe, and row-oriented ingestion using Snowpipe Streaming. Their sources, compute models, latency characteristics, and costs differ [6]. A data engineer selects an ingestion pattern only after agreeing on the event-to-reporting requirement.

1.6.2 Freshness and end-to-end latency metrics

Define *freshness* as how recently published data reflects the source, and *latency* as the elapsed time across identified pipeline boundaries. If a sensor reports at 09:00 and the curated table becomes queryable at 09:08, the observed source-to-table delay is eight minutes.

If the BI dashboard does not refresh until 09:30, the user's effective delay is thirty minutes. Measuring only ingestion hides the true result.

A proposed service-level objective (SLO) for a daily dashboard might be "the previous UTC day's validated metrics are available by 08:00 Europe/Helsinki, on 99% of scheduled days." For a near-live view, an SLO might instead target the 95th percentile of event-to-serving delay. These are *illustrative targets*; source limitations and stakeholder needs must determine the real ones.

1.6.3 Cost, complexity, and latency trade-offs

Pattern	Likely benefit	Operational burden
Scheduled batch	Simple replay and reconciliation	Delayed availability between runs
Frequent micro-batch	More frequent publication	More scheduling and file overhead
Continuous rows	Lower ingestion delay potential	Delivery state, monitoring, paid service footprint

Decision checkpoint. Would a 15-minute refresh meet the user's actual need? If yes, start there and test it before paying for a more elaborate architecture. If not, quantify what faster results change and design for backpressure, duplicates, and recovery.

Failure to avoid. Advertising "five-second real time" because a vendor documents a best-case ingestion figure. Benchmark the full path including external API, validation, transformations, BI refresh, and users' region [6].

Explore the source project: P007: Time-Series and Streaming Guide [GitHub]. *External reference; not executed or licensed for reproduction as part of this chapter.*

Explore the source project: P008: Recommender Pipeline [GitHub]. *External reference; not executed or licensed for reproduction as part of this chapter.*

1.7 When Snowflake is (and is not) the right choice

1.7.1 Strengths in BI, ELT, and governed sharing

Snowflake is worth considering when multiple teams need governed analytical data, a SQL-centric transformation workflow, independently configurable compute, and managed infrastructure rather than self-operated database servers. It can reduce infrastructure work and support collaboration around shared analytical objects. These are capabilities, not proof of cost or speed superiority for a specific workload [1, 5].

For a team already using SQL and BI tools, an attractive design may be to ingest raw observations, transform them in Snowflake, and publish reusable metrics. Such a design still depends on good dimensional modeling, tested definitions, and clearly assigned ownership.

1.7.2 Limitations, pricing, and vendor lock-in

Costs require a workload model. Warehouse compute, serverless services, storage, transfers, and specialized features can be billed differently. A virtual warehouse billed while active is not equivalent to a free always-on process; storage and region choices also matter [2, 7]. Avoid publishing a fixed price per run without date, account assumptions, and measurement.

Operational dependencies matter too. Proprietary SQL features, security policies, managed orchestration, and some platform-specific AI functionality can increase migration cost. A customer may reasonably choose another platform when transactional latency, open multi-engine access, existing infrastructure, data sovereignty, or specific budget constraints dominate.

1.7.3 Fair comparison criteria for alternatives

Compare products using *the same* data volume, source conditions, transformation semantics, concurrency, security constraints, recovery objectives, and cost window. Specify which systems handle metadata and storage, which execute computation, and who must operate each component. Never compare a fully warmed cached query with an uncached competitor's query and call the difference a platform result.

Architecture decision record — a fair beginning
Context: daily Finnish energy reporting with a small team. **Options:** SQL warehouse, open-table lakehouse, or managed analytics platform. **Criteria:** reliable source integration, reproducibility, governance, BI compatibility, staffing, monthly credits, and exit strategy. **Decision:** not yet final. Prototype the smallest adequate design, measure it, and record why alternatives were rejected.

Failure to avoid. Buying capacity or complex streaming infrastructure before documenting a freshness requirement. In engineering, a simple solution that demonstrably meets a contract is often superior to an impressive architecture with unbounded maintenance obligations.

Explore the source project: P009: Advanced Data Engineering with Snowflake [GitHub]. *External reference; not executed or licensed for reproduction as part of this chapter.*

1.8 Introducing the Finland Energy Analytics capstone

1.8.1 Stakeholders and business problem

We will build a *Finland Energy Analytics Platform*: a documented, testable data product that transforms licensed public energy observations into interpretable metrics. This is a teaching design, not an official Finnish energy statistic or a live production service. A system operator may care about timeliness; an analyst may care about comparability across years; a researcher may need provenance and reproducibility. Each needs a different kind of confidence in the same pipeline.

Before collecting data, define a narrow decision: *How did the composition and amount of reported electricity generation change over a chosen period?* We must select sources according to available variables and permitted reuse, rather than assume every public API exposes the desired measurements.

1.8.2 Energy generation/consumption indicators and time grain

A metric catalogue prevents arithmetic from becoming disguised business logic. At minimum, specify metric name, source, unit, grain, aggregation rule, missing-value behavior, and timezone. Power (MW) is not energy (MWh); power averaged over a time interval must be multiplied by that interval's duration in hours to obtain an energy contribution.

For time-series comparison, preserve original UTC timestamps and convert to local presentation time only with explicit daylight-saving handling.

Indicator	Unit / grain	Definition / caution
Mean power	MW / 15-min	Average reported power for each valid interval
Interval energy	MWh / 15-min	Mean MW multiplied by 0.25 h, *if* sample is an interval mean
Source freshness	Minutes / source	Publication time minus latest accepted event time
Completeness	Percent / day	Expected intervals received and accepted

1.8.3 Project roadmap across the book

The first milestone is a specification, not a database. Chapters 3–6 will establish an environment and load/normalize sources; Chapters 7–10 will compare and implement transformations; Chapters 11–13 will deliver metrics, governance, and operations; Chapter 14 will explore carefully evaluated AI extensions; Chapter 15 will integrate and assess the end-to-end result.

Scope boundary: In Chapter 1 all figures and sample readings are synthetic. Real Finnish data will be selected after verifying each provider's available variables, API terms, citation requirements, and download limits. The case study does not claim live grid coverage or forecasts.

Explore the source project: P010: Predictive Grid Maintenance [GitHub]. *External reference; not executed or licensed for reproduction as part of this chapter.*

Explore the source project: P011: Green-Energy IoT Engineering [GitHub]. *External reference; not executed or licensed for reproduction as part of this chapter.*

1.L Guided lab: design the first data product

1.L.1 Prerequisites, inputs, permissions, and cost guardrails

Goal: turn the running scenario into three reviewable artifacts before creating a paid Snowflake account. Produce three files: `problem_statement.md`, `requirements.md`, and `architecture_context.svg`. Use the synthetic input below; no account, credential, warehouse, or live API is required.

Synthetic input assumptions
Each row represents *mean power* in MW for one 15-minute UTC interval. Rows are uniquely identified by the pair (timestamp, source). The values are invented solely to check a dimensional and arithmetic decision; they are not Fingrid or Statistics Finland measurements.

Listing 1: Synthetic sample for the offline lab (energy_sample.csv).

```
timestamp_utc,source,power_mw,minutes
2026-01-15T00:00:00Z,WIND,480,15
2026-01-15T00:15:00Z,WIND,500,15
2026-01-15T00:00:00Z,SOLAR,0,15
2026-01-15T00:15:00Z,SOLAR,0,15
```

1.L.2 Step-by-step specification

1. **Problem statement:** write one paragraph identifying the reader's question, decision, source categories, out-of-scope uses, and success criterion. Do not promise a live national statistic before identifying a source.
2. **Stakeholders:** specify an analyst, platform operator, and reviewer. Give each an explicit question and desired freshness.
3. **Metric catalogue:** define `interval_energy_mwh` as `power_mw` $\times$ `minutes`/60, under the mean-power assumption. Define completeness on an expected-interval denominator.
4. **Data contract:** require UTC timestamps, known units, positive duration, source IDs, uniqueness, and a decision about rejected records.
5. **Architecture:** draw source $\rightarrow$ ingest $\rightarrow$ raw $\rightarrow$ validated $\rightarrow$ curated metric $\rightarrow$ dashboard, adding trust boundaries and ownership.

Suggested first acceptance criteria: the metric formula is traceable; source and grain are unambiguous; duplicate (timestamp, source) keys are rejected; no private credentials appear in deliverables; and a reviewer can reproduce the synthetic calculation offline.

Operational guardrail: the chapter requires zero Snowflake credits. For later labs, establish a separate learning account, a least-privilege role, warehouse auto-suspension, budget alerts, and cleanup steps before running cloud services [7]. Budgets are notifications and monitoring tools, not a guaranteed hard ceiling for every cost category.

1.L Verify, review, and extend

1.L.3 A runnable offline check and expected result

Save the CSV from the previous page as `energy_sample.csv`. The following standard-library Python code checks uniqueness, positive durations, and the energy calculation. It has been designed to run without Snowflake; the included companion file is exercised separately from any cloud integration.

Listing 2: Synthetic energy validation (chapter01_lab.py).

```
import csv
from collections import defaultdict

energy = defaultdict(float)
seen = set()
with open("energy_sample.csv", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        key = (row["timestamp_utc"], row["source"])
        assert key not in seen, f"Duplicate: {key}"
        seen.add(key)
        minutes = int(row["minutes"])
        assert minutes > 0, "Invalid interval"
        energy[row["source"]] += float(row["power_mw"]) * minutes / 60
print({k: round(v, 2) for k, v in sorted(energy.items())})
```

Expected output for the *synthetic* file: `{'SOLAR': 0.0, 'WIND': 245.0}`. A duplicate `WIND` timestamp should fail the uniqueness assertion. The code does not check licensing, true signal validity, or whether values are interval averages; those are separate checks. Production validation should raise explicit errors rather than rely on assertions that Python can disable.

1.L.4 Exercises and scoring rubric

Foundation: explain why 480 MW + 500 MW is not itself 980 MWh. **Practitioner:** add a rejected row with duration zero and describe the failure. **Production-design question:** specify how you would replay late corrections without double counting. **Scoring:** requirements 25%, metric semantics 25%, test evidence 25%, security/cost/limitations 25%.

Reviewer's acceptance checklist

- A business decision and primary reader are stated clearly.
- The metric catalogue distinguishes MW from MWh and names its time grain.
- UTC timestamps, duplicate identity, missing data, and source rights are addressed.
- Synthetic results are reproducible and plainly labelled synthetic.
- The architecture records alternatives, a freshness objective, and a cost boundary.

Chapter summary. Platform selection begins with the workload and its definition of correctness. Snowflake provides managed storage, compute, and cloud services, but the engineer must design the contracts, quality checks, operating model, and evidence that make results trustworthy. Chapter 2 follows a query through Snowflake's architecture.

SELECTED TECHNICAL REFERENCES

[1] Snowflake, Key Concepts and Architecture.

[2] Snowflake, Understanding Overall Cost. [3] Snowflake, Iceberg External Volumes.

[4] Snowflake, Hybrid Tables. [5] Snowflake, Secure Data Sharing.

[6] Snowflake, Snowpipe and Ingestion Choices.

[7] Snowflake, Controlling Cost.

Documentation consulted 21 October 2026; recheck editions, regions, features, licenses, and pricing before publication.